Git & GitHub Mastery

From zero to confident

how Git really works

the daily loop with real screenshots

undo almost any mistake

branching, rebase, cherry-pick, bisect, tags

the full pull-request flow

25 interview Q&A

glossary

@rathoreadiitya

v2.0 · Updated 2026-06-26

*Never used Git beyond `git add . && git commit`? Start at **§2 The mental model**, then go top to bottom.*

Most people copy-paste Git commands until something breaks, then panic. This sheet fixes that, end to end. You get the mental model of how Git actually works, the daily loop, branching, the undo and rescue commands for when things go wrong, the power tools (rebase, cherry-pick, bisect, tags), the full GitHub collaboration flow, and a set of interview questions, each core command shown with a real screenshot of what you will see on your screen. Learn the daily loop first; reach for the rest as you need it.

Start here (the student minimum). If you only learn six commands, learn these: git status, git add, git commit, git push, git pull, and git switch. That is 90% of every day. Sections 1 to 7 are this core. Everything from section 9 onward (reset, revert, stash, reflog, rebase, cherry-pick, bisect) is gear you reach for only when you need it. Read the core first, skim the rest, and come back the day you actually hit it. Prepping for an interview? Jump to section 20.

Inside this sheet

1. Who this sheet is for

If you can make commits but freeze the moment there is a merge conflict, a wrong commit, or a "your branch is behind" message, this is for you. No deep internals. Just the commands you reach for every day, plus the handful that save you when something goes wrong, each with a picture of what it looks like in real life.

2. The mental model - git vs GitHub, and the three areas

First, clear up the one thing everyone mixes up. Git and GitHub are not the same:

- Git is a tool on your own computer that records the history of your project. It works fully offline. You could use Git for years and never touch GitHub.
- GitHub is a website that stores a copy of that history online, so you can back it up and work with other people. GitLab and Bitbucket do the same job.

Short version: Git is the history on your laptop. GitHub is the copy in the cloud that the whole team shares.

Analogy: Git tumhare laptop ki personal diary hai, har change usme likha jaata hai aur internet ke bina bhi chalti hai. GitHub ek shared Google Doc hai jahan us diary ki copy chadha dete ho, taaki backup rahe aur poori team ek hi cheez pe kaam kar sake.

Second, every change you make passes through three areas. Almost every Git command just moves a change from one area to the next:

Area	What it is	How a change gets here
Working directory	Your actual files as you edit them right now	You edit a file in your editor
Staging area (index)	The changes you have picked for the next commit	git add <file>
Repository (.git)	The saved, permanent history of commits	git commit

Analogy: Working directory tumhara desk hai jahan abhi kaam ho raha hai. Staging wo bag hai jisme tum decide karke cheezein daalte ho jo submit karni hain. Repository submitted assignment hai, permanently record ho gaya. git add desk se bag mein daalta hai, git commit bag ko submit kar deta hai.

So the daily loop is just: edit (working directory), git add (staging), git commit (repository), git push (send to GitHub). Two more terms you will see everywhere:

- HEAD is just a pointer to where you are right now, usually the latest commit on your current branch.
- A branch is a lightweight movable pointer to a commit, a separate line of work you can build on without touching the main code.

Analogy: Branch ek alag notebook hai. Main notebook (main branch) ko tum kharaab nahi karte. Apne feature ka kaam alag notebook mein karte ho, jab perfect ho jaaye tabhi main notebook mein laate ho. HEAD bas ungli hai jo batati hai abhi tum kaunse page pe ho.

3. How Git really works - commits, SHAs, the graph, HEAD and refs

You can use Git for years without this, but ten minutes here makes every later command obvious instead of magic. Git is not storing "the changes you made". It stores full snapshots of your project, linked in a chain.

A commit is a snapshot, not a diff

Each time you commit, Git takes a picture of every tracked file at that moment and saves it with a few facts: who made it, when, a message, and a link to the commit that came before it (its parent). String those parents together and you get your project's history.

- Every commit has a unique 40-character ID called a SHA (a hash like a1b2c3d...). You almost always use just the first 7 characters (a1b2c3d).
- A commit points to its parent, so history is a chain. A merge commit has two parents, which is how two lines of work join.
- Because of this chaining, the whole history is a directed graph (a DAG), not a simple list. That is what git log --graph draws.

Analogy: Har commit ek photo hai poore project ki, aur uske peeche likha hai pichli photo ka number (parent). Photos ki yeh chain hi tumhari history hai. SHA us photo ka unique barcode hai.

HEAD, branches, and tags are just pointers

Here is the part that makes everything else click. A branch is not a copy of your files. It is just a sticky note pointing at one commit. HEAD is a sticky note pointing at the branch you are currently on. Make a commit and Git simply moves the branch note forward to the new commit.

Pointer	What it points to	Moves when

Pointer	What it points to	Moves when
HEAD	the branch (and so the commit) you are on right now	you switch branches or commit
a branch (main, feature)	the latest commit on that line of work	you commit on it
a tag (v1.0)	one specific commit, frozen forever	never - tags do not move

This is why branching in Git is instant and cheap: making a branch just writes one tiny file with a commit ID in it. No copying. That is why branching beats the older tools that copied the whole project.

Pointing at commits - HEAD~ and HEAD^

Tons of commands take a commit. Instead of copying a SHA, you can step back from where you are:

Reference	Means
`HEAD`	the commit you are on right now
`HEAD~1  (or HEAD~)`	one commit before HEAD (its parent)
`HEAD~3`	three commits back
`HEAD^`	the first parent - matters only at a merge commit
`HEAD@{2}`	where HEAD was 2 moves ago (from the reflog, see section 9)
`a1b2c3d`	a commit by the first 7 chars of its SHA

Why this matters next: git reset HEAD~1, git rebase -i HEAD~3, and git show HEAD~2 all use this. Once HEAD~n clicks, half the scary commands stop being scary.

4. One-time setup (do this once per machine)

Set your name and email so your commits are labelled correctly. Do this once on any new computer.

Terminal

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Why the last line: it makes new repos start on a branch called main instead of master, which is what GitHub uses now.

Logging in to GitHub (you cannot push without this)

GitHub stopped accepting your account password for git push back in 2021. The first time you push, it asks you to prove who you are. Pick one of these two ways, set it up once, and you never think about it again.

Way to log in	How you set it up	Best when

Way to log in	How you set it up	Best when
HTTPS + token	On GitHub go to Settings, Developer settings, Personal access tokens, and generate one. When git asks for a password, paste the token instead of your real password. Git remembers it after that.	You are new and just want it to work. Simplest path.
SSH key	Run ssh-keygen, then copy the .pub key into GitHub Settings, SSH keys. Now clone using the SSH URL (git@github.com:you/repo.git).	You push a lot from your own machine and want no token prompts.

On a college or lab PC: the HTTPS + token way is safest. Do not leave an SSH key on a shared machine, and sign out of any credential manager when you are done.

5. Start a repo - init, clone, remote

Two ways to begin. Either turn a folder you already have into a repo, or copy an existing one from GitHub.

Terminal

```
# Option A: start fresh in the current folder
git init

# Option B: copy an existing repo from GitHub to your machine
git clone https://github.com/user/repo.git
```

If you started with git init and want to connect it to a GitHub repo you made, point it at the remote. Get the URL from GitHub: open your repo, click the green Code button, and copy the HTTPS URL.

Terminal

```
git remote add origin https://github.com/you/your-repo.git
git remote -v        # check which remotes are connected
```

origin is just a name: it is the default nickname for the main remote. You are not stuck with it, but everyone uses origin, so keep it.

6. The daily loop - status, add, commit, push, pull

This is 90% of what you do. Check what changed, stage it, commit it, send it up, and pull what others pushed.

Terminal

```
git status                  # what changed, what is staged
git add file.py             # stage one file
git add .                   # stage everything changed
git commit -m "message"     # save a snapshot to history
git push                    # send your commits to GitHub
```

```
git pull                    # bring down commits others pushed
```

git status is the command you run most. It shows three groups in colour: staged changes (green, ready to commit), changes not staged (red, edited but not added yet), and untracked files (red, brand new, Git is not watching them yet).

[Screenshot: git status with a staged file, an unstaged edit, and an untracked file]

drop screenshots/01-git-status.png here

Analogy: Har commit ek save point hai, jaise video game ka checkpoint. Aage jaake kuch bigad gaya? Us checkpoint pe waapas aa jao. Isliye chhote chhote commits karo, ek hi bade commit mein poora din mat thoso.

A commit message is a note to your future self. Make it say what changed, not that something changed:

Weak message	Strong message
update	Fix login button not responding on mobile
changes	Add input validation to signup form
final final v2	Refactor payment retry to back off on timeout

First push of a new branch: use git push -u origin branch-name. The -u sets the upstream once, so every later push is just git push with nothing extra to type.

git pull is not one action. It is git fetch (download the new commits) followed by git merge (join them into your branch). Knowing that is why pull can cause a merge conflict.

If main moved ahead while you worked: git pull merges the new commits in and may add a merge commit. git pull --rebase instead replays your commits on top of the latest main, keeping history a clean straight line. Solo or small projects: either is fine. Prefer --rebase when you want a tidy log.

7. Branching - switch, merge, delete

A branch lets you build a feature without touching working code. Make one, do your work, then merge it back into main.

Terminal

```
git switch -c feature-login   # create a branch and move onto it
git switch main               # go back to main
git branch                    # list branches, * marks current
git merge feature-login       # merge that branch into the current one
git branch -d feature-login   # delete it after merging
```

Older syntax you will still see: git checkout -b feature-login does the same as git switch -c, and git checkout main is the same as git switch main. switch is the newer, clearer command. Both work.

git log --oneline --graph --all draws the branch structure in the terminal so you can see where branches split and merged:

[Screenshot: git log --oneline --graph --all showing a branch and a merge commit]

drop screenshots/02-log-graph.png here

Analogy: Merge do logon ka kaam jodna hai. Dono ne alag alag lines likhi, Git khud jod deta hai. Par dono ne theek same line ko alag tareeke se badla? Tab Git ruk jaata hai aur bolta hai 'tum decide karo kaunsa rakhna hai'. Usi ko conflict kehte hain.

8. Inspect - log, diff, show, blame

Before you commit, look at exactly what you are about to save.

Terminal

```
git log --oneline      # compact history, one line per commit
git diff               # changes NOT yet staged (working vs staged)
git diff --staged      # changes that ARE staged (staged vs last commit)
git show <commit>      # everything a specific commit changed
```

The diff trap: git diff shows nothing after you git add, because the change moved to staging. To review staged changes use git diff --staged. This confuses everyone once.

Same edit, two views. Unstaged on the left of your workflow, staged on the right:

[Screenshot: git diff (unstaged) next to git diff --staged on the same edit]

drop screenshots/03-diff.png here

log, but useful - the flags worth knowing

Command	What it shows
`git log --oneline --graph --all`	the whole branch structure as a graph
`git log -5`	just the last 5 commits
`git log --stat`	each commit plus which files changed and how much
`git log -p`	each commit with the full diff inline
`git log --author="Aditya"`	only commits by one person
`git log --since="2 weeks ago"`	only recent commits
`git log -- file.py`	the history of one file only

Who wrote this line, and why - git blame

When you find a strange line and want to know who added it and in which commit (so you can read that commit's message and reasoning), blame is the tool. It is not about blaming people, it is about finding context.

Terminal

```
git blame file.py            # every line: commit, author, date
git blame -L 20,30 file.py   # only lines 20 to 30
```

Analogy: Blame har line ke aage uska 'kisne aur kab likha' likh deta hai. Koi line samajh nahi aa rahi? Blame se uss commit ka address lo, phir git show se uss commit ka message padho - wahin reason milta hai.

Compare two branches

Before you merge or open a PR, see exactly how your branch differs from main:

Terminal

```
git diff main feature        # every line that differs between the two
git diff main...feature      # only what feature added since it split off
git diff main --stat         # just the list of changed files, no line detail
```

9. Undo and rescue - restore, reset, wrong branch, revert, stash, reflog, clean

The commands that save you. Match the situation to the command.

Throw away edits you have not committed

Terminal

```
git restore file.py        # discard unstaged edits in a file
git restore --staged file.py   # unstage, but keep your edits
git restore --source=HEAD~2 file.py  # bring back this file as it was 2 commits
ago
```

Undo a commit with reset (three levels)

reset moves your branch pointer back. The flag decides how much it throws away. This table is the whole point, read the last two columns:

Command	Your commit	Your file edits	Use when

Command	Your commit	Your file edits	Use when
git reset --soft HEAD~1	undone	kept and still staged	you want to redo the commit message or add one more file
git reset HEAD~1 (default)	undone	kept but unstaged	you want the commit gone but your work back in progress
git reset --hard HEAD~1	undone	deleted, gone	you want the commit AND the changes thrown away completely

--hard deletes uncommitted work with no undo prompt. Run git status first to be sure you have nothing you want to keep.

Analogy: Soft reset matlab commit undo, par kaam staged hi rehne diya (bas message dobara likhna hai). Mixed (default) matlab undo, kaam wapas unstaged (phir se sochna hai). Hard matlab commit aur kaam dono uda diye (pakka ho tabhi karo).

After a hard reset, brand-new files stay: git reset does not touch untracked files. To wipe those too, use git clean -fd (this permanently deletes them, so run git clean -nd first to preview what it would remove).

You committed on the wrong branch

Made a few commits on main that were meant for a feature branch? Very common when working alone. Move them without losing anything:

Terminal

```
git switch -c feature-x      # make the right branch from where you are
git switch main              # go back to main
git reset --hard HEAD~3      # rewind main past your 3 stray commits
# the commits now live only on feature-x, main is clean again
```

Only if not pushed yet: this rewrites main. If you already pushed those commits to a shared main, use git revert instead (see below).

Undo a commit that is already pushed - use revert

Never reset a commit you already pushed to a shared branch. Instead, revert makes a NEW commit that cancels out the old one. History stays intact, and nobody else's copy breaks.

Terminal

```
git revert <commit>    # safe undo on a shared branch
```

Analogy: Reset notebook se page hi phaad deta hai, jaise wo kabhi tha hi nahi. Revert page rehne deta hai par uspe cross maar deta hai, taaki sabko dikhe kya tha aur kyun hataya. Team ke saamne wali history mein revert hamesha safe hai.

Park your work to deal with something urgent - stash

Terminal

```
git stash          # shelve your uncommitted changes, clean the slate
git stash pop      # bring them back when you are ready
git stash list     # see what you have shelved
```

Analogy: Stash matlab adha kaam bookmark karke side mein rakh diya, kisi urgent cheez pe jao, baad mein aake wahin se uthaa lo. Jaise Netflix ka 'watch later', pause karke kabhi bhi finish kar sakte ho.

Recover something you thought you lost - reflog

reflog is the time machine. It logs everywhere HEAD has been, even after a bad reset. Find the point you want and reset back to it.

Terminal

```
git reflog                    # list recent HEAD positions
git reset --hard HEAD@{2}     # jump back to where you were 2 moves ago
```

[Screenshot: git reflog output showing the HEAD@{n} history you can return to]

drop screenshots/04-reflog.png here

Analogy: Reflog Git ka CCTV hai. HEAD jahan jahan gaya, sab record hota hai. Galti se hard reset maar ke kaam uda diya? Ghabrao mat, CCTV se uss point ka address (HEAD@{n}) dekho aur wahin reset kar ke wapas aa jao.

Fix a merge conflict

When a merge or pull hits the same lines two ways, Git pauses and marks the file. Open it, decide whether to keep your version, the incoming version, or a combination of both, delete all three marker lines (<<<<<<<, =======, >>>>>>>), then add and commit.

Conflict markers in the file

```
<<<<<<< HEAD
your version of the line
=======
the incoming version of the line
>>>>>>> feature-branch
```

Terminal

```
# after editing the file to keep what you want and deleting the markers
git add file.py
git commit          # completes the merge
```

[Screenshot: a merge conflict in the terminal, then the file with markers resolved]

drop screenshots/05-conflict.png here

Too messy and you just want out? git merge --abort throws away the half-done merge and puts you back exactly where you were before you ran merge. Nothing lost. Catch your breath and try again.

10. Rewriting history - amend, rebase, interactive rebase, cherry-pick + the golden rule

These commands change commits that already exist. They are powerful and safe on your own un-pushed work, and dangerous on shared history. Learn the golden rule at the end before you use any of them on a team branch.

Fix the last commit - amend

Terminal

```
git commit --amend                  # change the last commit's message
git add forgot.py && git commit --amend --no-edit  # add a forgotten file to it
```

Use it for: the classic "oops, typo in the message" or "forgot to add one file", right after committing and before pushing.

Merge vs rebase - the question interviewers love

Both bring the latest main into your feature branch, but they shape history differently:

	git merge main	git rebase main
What it does	joins the two lines with a merge commit	replays your commits on top of the latest main
History	keeps the true, branching shape	becomes one straight line
Safe on shared branch?	yes	no - it rewrites your commits
Best for	preserving exactly what happened	a clean, linear log before opening a PR

Analogy: Merge do raaste jod deta hai aur junction (merge commit) chhod deta hai, history mein dono raaste dikhte hain. Rebase tumhare kaam ko utha ke latest main ke upar dobara

rakh deta hai, jaise tumne shuru se wahin kaam kiya ho - history ek seedhi line ban jaati hai.

Clean up your commits - interactive rebase

Before opening a pull request, you often want to tidy a messy series of commits ("wip", "fix", "fix again") into a few clean ones. Interactive rebase lets you do that.

Terminal

```
git rebase -i HEAD~4    # edit the last 4 commits interactively
```

Git opens a list in your editor that looks like this. You change the word in front of each commit, then save and close:

rebase editor

```
pick a1b2c3d feat: add login form
squash e4f5g6h wip
squash i7j8k9l fix typo
reword m0n1o2p add validation
# Save and close the editor to run it.
```

You set the action in the first column to one of these:

Action	What it does
`pick`	keep the commit as-is
`reword`	keep the commit, change its message
`squash`	merge this commit into the one above, combine messages
`fixup`	like squash but throw this commit's message away
`drop`	delete the commit entirely
`edit`	pause here so you can amend the commit

Most common use: turn five "wip" commits into one tidy commit by setting the first to pick and the rest to fixup.

If a rebase hits a conflict it pauses. Fix the conflicted file (same <<<<<<< markers as a merge), run git add file, then git rebase --continue to carry on. Stuck or scared? git rebase --abort puts everything back exactly as it was before you started.

Squash a whole branch into one commit - merge --squash

Want to merge a feature branch but land it as a single tidy commit on main instead of all its little commits plus a merge commit? Use squash merge. It applies the changes but does not commit, so you write one clean commit yourself.

Terminal

```
git switch main
git merge --squash feature
```

```
git commit -m "Add login feature"   # one clean commit for the whole branch
```

On GitHub: the "Squash and merge" button in a pull request does exactly this for you. Most teams use it to keep main's history clean.

Grab one commit from another branch - cherry-pick

Need just one specific commit (a bug fix) from another branch, without merging the whole thing? Cherry-pick copies that single commit onto your current branch.

Terminal

```
git cherry-pick <commit>          # apply that one commit here
git cherry-pick <commitA> <commitB>   # apply several, in order
```

Analogy: Cherry-pick matlab doosri branch se sirf ek commit utha ke apni branch pe chipka diya - poori branch merge kiye bina. Jaise thaali se sirf ek mithai uthana.

The golden rule of Git: never amend, rebase, or hard-reset a commit you have already pushed to a branch other people use. It rewrites history that others already have, and it breaks their copy. Rewrite freely on your own un-pushed work, never on shared history.

If you must push rewritten history (your own branch only): use git push --force-with-lease, not git push --force. --force-with-lease refuses to overwrite the remote if someone else pushed in the meantime, so you never silently wipe a teammate's work. Plain --force has no such safety check.

11. Find what broke - git bisect

Something works today that worked last week, but you have 80 commits in between and no idea which one broke it. bisect finds the exact commit by binary search, in about 7 steps instead of 80.

Terminal

```
git bisect start
git bisect bad                # the current commit is broken
git bisect good v1.0          # this old commit/tag was fine
# git checks out a commit halfway between. You test it, then say:
git bisect good               # ...if this one works
git bisect bad                # ...if this one is broken
# repeat until Git names the first bad commit, then:
git bisect reset              # go back to where you started
```

Analogy: Bisect 'number guessing game' jaisa hai. Tum bolte ho yeh commit kharaab, woh theek tha; Git beech wala commit deta hai test karne. Har step aadhe commits cut, isliye 80

commits sirf 7 tests mein chhant jaate hain.

Pro move: if you have a test script that exits 0 on good and non-zero on bad, run git bisect run ./test.sh and Git finds the bad commit fully automatically.

12. Tags and releases - versioning your code

A tag is a permanent label on one commit, used to mark released versions (v1.0, v2.3.1). Unlike a branch, a tag never moves.

Terminal

```
git tag v1.0                          # lightweight tag on the current commit
git tag -a v1.0 -m "First release"  # annotated tag (preferred: has message + author)
git tag                               # list tags
git push origin v1.0                  # tags are NOT pushed by default - push it
git push origin --tags                # push all tags
```

Version numbers usually follow semantic versioning, MAJOR.MINOR.PATCH:

Part	Bump it when	Example
MAJOR	you make a breaking change	1.4.2 -> 2.0.0
MINOR	you add a feature, nothing breaks	1.4.2 -> 1.5.0
PATCH	you only fix a bug	1.4.2 -> 1.4.3

On GitHub: a pushed tag can be turned into a Release with notes and downloadable files, under the repo's Releases tab. Great for showing finished project versions on your portfolio.

13. Remotes in depth - fetch vs pull, tracking, prune, many remotes

A remote is just a nickname for a repo URL somewhere else (usually GitHub). origin is the default one. Here is what you actually do with them.

Terminal

```
git remote -v                           # list remotes and their URLs
git remote add upstream <url>           # add a second remote (e.g. the original you forked)
git remote set-url origin <new-url>     # change a remote's URL
git fetch origin                        # download new commits, do NOT merge
git branch -r                           # list remote-tracking branches
```

fetch vs pull - the difference that trips everyone

git fetch downloads the latest commits from the remote but does not touch your working files. git pull is fetch plus an automatic merge into your branch. Fetch when you want to look before you leap; pull when you are ready to integrate.

git pull = git fetch + git merge. That is the whole secret. Knowing it is exactly why a pull can suddenly throw a merge conflict.

Tracking branches and pruning dead ones

Terminal

```
git push -u origin feature      # link local feature to origin/feature (do once)
git fetch --prune               # delete local refs to remote branches that are
gone
git branch -d feature           # delete a local branch after it is merged
```

Why prune: after PRs get merged and their branches deleted on GitHub, your machine still lists the old origin/feature names. git fetch --prune cleans them up.

14. GitHub - fork vs clone, and the pull-request flow

First, two words students swap by mistake:

- Fork makes your own copy of someone else's repo on GitHub (server side). You use it to contribute to a project you do not own.
- Clone copies a repo from GitHub down to your computer (local side). You use it to work on the code.

Analogy: Fork matlab GitHub pe kisi aur ki repo ka apna copy bana liya (server pe, original se juda hua). Clone matlab us repo ko apne laptop pe utaar liya. Open source mein contribute karna ho toh pehle fork, phir apne fork ko clone.

The standard team flow, the one you will do at every internship:

1. Make a branch and push it: git switch -c feature, then git push -u origin feature.
2. On GitHub, a "Compare & pull request" button appears. Click it to open a pull request (a PR).
3. The PR shows your changes and lets teammates review and comment line by line.
4. Once approved, click Merge pull request (or Squash and merge) to fold it into main.

What that button looks like right after you push:

[Screenshot: the green Compare & pull request button on GitHub after a push]

drop screenshots/06-compare-pr.png here

Inside an open pull request, the Files changed tab and a review comment on a line:

[Screenshot: an open PR showing the diff and a line-level review comment]

drop screenshots/07-pr-review.png here

The banner that tells you your branch is ahead of or behind main:

[Screenshot: the This branch is N commits ahead banner on a GitHub branch]

drop screenshots/08-branch-ahead.png here

The repo's commit history, so you can see the project's timeline:

[Screenshot: the commit history list on GitHub]

drop screenshots/09-commit-history.png here

And the merge options when the PR is approved:

[Screenshot: the Merge pull request button with the Squash and merge dropdown open]

drop screenshots/10-merge-dropdown.png here

Working alone on a college project, with nobody to review? Use the same branch and merge workflow to keep main clean, but skip the pull request. Once your feature branch works, git switch main then git merge feature and push. Open a PR only when there is actually someone to review it.

15. GitHub collaboration - issues, syncing a fork, reviews, Actions, gh CLI

The pull request is the core, but a real project on GitHub has a few more moving parts you will run into fast.

Issues - the to-do list and bug tracker

Issues are where work and bugs are tracked and discussed. Type "Fixes #42" in a PR description and GitHub auto-closes issue 42 when the PR merges. Link your commits to issues this way so reviewers see the context.

Keep your fork in sync with the original

When you fork a project and work for a few days, the original (upstream) moves ahead. The easy way: open your fork on GitHub and click the "Sync fork" button. The command-line way, which you should also know:

Terminal

```
git remote add upstream <original-repo-url>   # do this once; copy the URL
                                              # from the original repo's Code
button
git fetch upstream
git switch main
git merge upstream/main        # now your fork's main is up to date
git push origin main           # update your fork on GitHub too
```

Analogy: origin tumhara fork hai, upstream original repo hai. Original aage badhta rehta hai; upstream se fetch + merge karke apne fork ko uske saath sync rakhte ho, taaki tumhara PR purana na pad jaaye.

Reviews, draft PRs, and protected branches

- Draft PR: open a pull request marked "draft" to share work-in-progress and get early feedback before it is ready to merge.
- Review states: a reviewer can Approve, Comment, or Request changes. Most teams need at least one approval before merge.
- Protected branch: main is often locked so nobody can push directly. You must go through a PR that passes review and checks. This is normal and good.

Checks that run automatically - GitHub Actions

GitHub Actions runs tasks (tests, linting, builds) automatically on every push or PR. You will see green checks or red X's on a PR. Red means a check failed and you should fix it before merging. The config lives in .github/workflows/ as YAML files.

The GitHub CLI - gh

Do GitHub things from the terminal without opening the browser:

Terminal

```
gh auth login            # sign in once
gh repo clone owner/repo # clone
gh pr create             # open a PR from your current branch
gh pr status             # see your PRs at a glance
gh pr checkout 123       # check out PR #123 locally to test it
```

16. .gitignore, .gitattributes + the Windows line-ending trap

A .gitignore file lists what Git should never track: build folders, secrets, node_modules, .env files. Add it before your first commit so junk never enters history.

.gitignore

```
# .gitignore
node_modules/
__pycache__/
.env
*.log
dist/
```

Do not write it from scratch: github.com/github/gitignore has ready-made templates for every language. Copy the one for your stack.

Already committed something you should have ignored (like node_modules or a .env)? Adding it to .gitignore does nothing on its own. Git is already tracking it. Stop tracking it with git rm -r --cached node_modules then commit. The files stay on your disk, they just leave Git.

Two common team workflows. For a student or a small team, feature-branch is enough.

Workflow	How it works	Use it when
Feature-branch	main is always working code. Each feature gets a branch, then a PR back into main.	Almost always, including college projects and small teams
Gitflow	Separate long-lived branches for develop, release, and hotfix on top of main.	Bigger teams shipping scheduled releases, often overkill for students

The Windows line-ending trap - CRLF vs LF

Windows ends lines with two invisible characters (CRLF); Mac and Linux use one (LF). On mixed teams this makes Git think every line changed even when nobody touched them, flooding your diff. Fix it once with config:

Terminal

```
git config --global core.autocrlf true    # on Windows
git config --global core.autocrlf input   # on Mac/Linux
```

Better, commit a .gitattributes file so the rule travels with the repo for everyone, no matter their machine:

.gitattributes

```
# .gitattributes
* text=auto
*.sh text eol=lf
*.bat text eol=crlf
*.png binary
```

Why bother: .gitattributes is checked into the repo, so it normalises line endings for every contributor automatically. core.autocrlf only fixes your own machine.

17. Power-ups - aliases, commit conventions, hooks, config

Small touches that make you look (and work) like a pro.

Aliases - shortcuts for commands you type all day

Terminal

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.lg "log --oneline --graph --all"
# now: git st, git co, git lg
```

Conventional Commits - messages with a pattern

Many teams write commit messages in a fixed format so history reads cleanly and tools can auto-generate changelogs. The shape is type: summary.

Prefix	Use for
`feat:`	a new feature
`fix:`	a bug fix
`docs:`	documentation only
`refactor:`	code change that is neither a feature nor a fix
`test:`	adding or fixing tests
`chore:`	build, tooling, dependencies

Example: feat: add OTP login or fix: handle empty cart crash

Hooks - run something automatically on commit

Git hooks are scripts that fire on events like committing. A pre-commit hook can run your linter or tests and block the commit if they fail. Most teams set these up with a tool like pre-commit (Python) or husky (JavaScript) rather than by hand, so the hooks are shared with everyone.

Config worth knowing

Terminal

```
git config --global pull.rebase true     # keep history linear on pull
git config --global init.defaultBranch main
git config --global core.editor "code --wait"   # use VS Code for messages
git config --list                        # see everything that is set
```

18. The 20 commands you actually use (revision card)

Memorise these and you can handle almost any day.

Command	What it does
`git status`	show what changed and what is staged
`git add .`	stage all your changes
`git commit -m "..."`	save a snapshot to history
`git push`	send commits to GitHub
`git pull`	fetch and merge commits others pushed
`git fetch`	download new commits without merging
`git clone <url>`	copy a repo to your computer
`git switch -c name`	create a branch and move onto it
`git switch main`	go back to the main branch
`git merge name`	merge a branch into the current one
`git rebase main`	replay your branch on top of main
`git log --oneline --graph`	see history and branch structure
`git diff`	see unstaged changes
`git stash`	shelve changes to come back to later
`git restore file`	discard unstaged edits in a file
`git reset --soft HEAD~1`	undo the last commit, keep the work staged
`git revert <commit>`	safely undo a commit that is already pushed
`git cherry-pick <commit>`	copy one commit onto the current branch
`git tag -a v1.0 -m "..."`	mark a release version
`git reflog`	the safety log - find lost commits

19. If you see this error, run this

The message you see	What it means + what to run
fatal: not a git repository	You are not inside a repo. cd into your project, or run git init.
Updates were rejected ... fetch first	GitHub has commits you do not. Run git pull, resolve any conflict, then git push.
CONFLICT ... Merge conflict in <file>	Open the file, fix the <<<<<<< markers, then git add and git commit.

The message you see	What it means + what to run
You are in detached HEAD state	You checked out a commit instead of a branch (like git checkout abc123). Any new commit here gets orphaned once you leave. To keep your work run git switch -c new-branch; to just leave run git switch main.
Please tell me who you are	Set your identity: git config --global user.email and user.name.
Permission denied (publickey)	Your SSH key is not set up. Use the HTTPS repo URL, or add an SSH key to GitHub.
error: failed to push some refs	Usually the same as 'rejected, fetch first'. Run git pull --rebase, fix any conflict, then push again.
Your branch and origin/main have diverged	Both sides have new commits. git pull (merge) or git pull --rebase to combine them, resolve conflicts, then push.
Please commit your changes or stash them before you merge	You have uncommitted edits in the way. git stash, do the merge/pull, then git stash pop.
fatal: refusing to merge unrelated histories	Two repos with no shared start. Add --allow-unrelated-histories to your merge or pull, once, on purpose.

20. Interview questions - 25 answers ready to say out loud

These come up constantly in internship and new-grad interviews. Each answer is one or two lines you can actually speak.

1. What is a commit?

A snapshot of all tracked files at a moment in time, with a message, an author, and a link to its parent commit. Commits chained by parents form your history.

2. What is the difference between Git and GitHub?

Git is the version-control tool that runs on your machine; GitHub is one cloud host for Git repositories. The key point: Git does not need GitHub. You could host the same repo on GitLab, Bitbucket, or your own server instead.

3. What are the three areas in Git?

Working directory (your files), staging area (changes marked for the next commit via git add), and the repository (committed history). git commit moves staged changes into history.

4. What is HEAD?

A pointer to the commit you are currently on, usually through the branch you have checked out. Move HEAD by switching branches or committing.

5. What is a detached HEAD?

When HEAD points straight at a commit instead of a branch, usually because you checked out a specific commit. New commits there are not on any branch and get lost unless you make a branch to keep them.

6. Difference between git merge and git rebase?

Merge joins two branches and creates a merge commit, keeping the true history. Rebase replays your commits on top of another branch for a clean, linear history. Never rebase commits others already have.

7. Difference between git fetch and git pull?

git pull = git fetch + git merge. Fetch only downloads, leaving your files untouched so you can inspect first; pull also merges, which is why a pull can suddenly raise a conflict. git pull --rebase merges by rebasing instead.

8. Difference between git reset and git revert?

Reset moves the branch pointer back and can rewrite history, so it is for local un-pushed work. Revert creates a new commit that undoes an old one, so it is safe on shared, pushed history.

9. What is the difference between soft, mixed, and hard reset?

Soft keeps your changes staged, mixed keeps them as unstaged edits (the default), and hard throws the changes away entirely. Hard is destructive.

10. What is git stash?

It shelves your uncommitted changes so your working directory is clean, lets you do something else, then git stash pop brings them back. Handy when you must switch branches mid-task.

11. What is a fast-forward merge?

When the target branch has no new commits, Git just moves its pointer forward to your branch's tip. No merge commit is needed. Use git merge --no-ff to force a merge commit anyway.

12. What does git cherry-pick do?

It copies one specific commit from another branch onto your current branch, without merging the whole branch. Good for grabbing a single bug fix.

13. What is origin?

The default nickname for the remote repository you cloned from. It is just a shortcut for a URL, not a special word.

14. Difference between origin and upstream?

By convention, when you fork a project, origin is your fork and upstream is the original repo. You fetch from upstream to stay in sync and push to origin.

15. What is a fork versus a clone?

A fork is a server-side copy on GitHub; a clone is a local copy on your machine. You fork because you cannot push to a repo you do not own, so you make your own copy, clone that, push to it, then open a PR back to the original.

16. How do you resolve a merge conflict?

Open the file, find the <<<<<<< ======= >>>>>>> markers, edit it to the final version you want, delete the markers, then git add the file and commit to finish the merge.

17. You added *.log to .gitignore but Git still tracks your log file. Why?

.gitignore only stops Git from tracking NEW, untracked files. A file already committed stays tracked. You must stop tracking it with git rm --cached file (it stays on disk) and commit, then .gitignore takes effect.

18. What is git bisect?

A binary-search tool to find the commit that introduced a bug. You mark a known good and bad commit and Git keeps checking out the midpoint until it pinpoints the first bad one.

19. What is the reflog?

A local log of everywhere HEAD has been, even after resets or rebases. It is your undo of last resort: git reflog finds a 'lost' commit's SHA so you can recover it.

20. Why can't you rebase a commit you already pushed?

Rebase gives each commit a new SHA, so your history no longer matches what teammates already pulled. When they pull, the two histories collide into a mess, and you would need a force-push that can wipe their work. That is the golden rule: never rewrite shared history.

21. What does git commit --amend do?

It replaces the last commit, letting you fix its message or add a forgotten file. Only do it before pushing, since it rewrites that commit.

22. What is a tag, and how is it different from a branch?

Both are pointers to a commit, but a branch pointer moves forward every time you commit, while a tag stays frozen on one commit. Use branches for ongoing work and tags to mark release versions like v1.0.

23. What is an interactive rebase used for?

git rebase -i lets you reorder, reword, squash, or drop your recent commits to clean up history before opening a pull request.

24. What is a pull request?

A GitHub request to merge your branch into another, where teammates review the diff and comment before it is merged. It is the standard way teams ship and review changes.

25. How do you undo the last commit but keep your changes?

git reset --soft HEAD~1 removes the commit but leaves the changes staged, ready to recommit. Use --mixed to keep them as unstaged edits instead.

21. Glossary - every term on this sheet in one line

Term	Meaning
`Repository (repo)`	a project folder that Git is tracking, with its full history
`Commit`	a saved snapshot of your tracked files at one moment
`SHA / hash`	the unique 40-character ID of a commit; you use the first 7
`Working directory`	your actual files, where you edit
`Staging area / index`	the holding zone for changes going into the next commit
`Branch`	a movable pointer to a line of work
`HEAD`	a pointer to the commit/branch you are currently on
`main`	the default primary branch
`Tag`	a fixed label on one commit, used for releases
`Remote`	a nickname for a repo hosted elsewhere, like GitHub
`origin`	the default name for the remote you cloned from
`upstream`	by convention, the original repo you forked from
`Clone`	make a local copy of a remote repo
`Fork`	make your own server-side copy of someone else's repo
`Fetch`	download remote commits without merging
`Pull`	fetch plus merge into your branch
`Push`	send your commits to a remote
`Merge`	combine another branch into the current one
`Fast-forward`	a merge done by just moving the pointer, no merge commit

Term	Meaning
`Rebase`	replay your commits on top of another branch for a linear history
`Cherry-pick`	copy a single commit onto the current branch
`Conflict`	when two changes touch the same lines and Git needs you to choose
`Stash`	temporarily shelve uncommitted changes
`Reset`	move the branch pointer back; can discard commits
`Revert`	make a new commit that undoes an earlier one
`Restore`	discard edits or unstage a file
`Amend`	replace the most recent commit
`Detached HEAD`	HEAD on a commit instead of a branch; new work can be lost
`Reflog`	local log of where HEAD has been; used to recover lost commits
`Bisect`	binary search across commits to find what broke
`Pull request (PR)`	a GitHub request to merge and review a branch
`.gitignore`	list of files Git should never track
`.gitattributes`	per-repo rules, e.g. line-ending normalisation

22. Final word

In your first weeks you will use just the daily loop: status, add, commit, push, pull. Everything else on this sheet is rescue gear and team-flow knowledge you pull out the day you need it, and now you know exactly which one to reach for. Git stops being scary the moment you see that most commands just move a change between three areas. Keep the revision card and glossary handy, skim the interview questions before an interview, and walk into your next project knowing you can recover from almost any mistake.